```
import pandas as pd
import numpy as np
import nltk
import re
from bs4 import BeautifulSoup
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import Perceptron, LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

nltk.download('wordnet')
nltk.download('stopwords')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
True
```

```
%pip install bs4

# Dataset: https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_Beauty_v1_00.tsv.gz
#          https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_Office_Products_v1_00.tsv.gz
```

```
Collecting bs4
  Downloading bs4-0.0.2-py2.py3-none-any.whl.metadata (411 bytes)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from bs4) (4.13.5)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->bs4) (2.8.1)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->bs4) (4.15.0)
Downloading bs4-0.0.2-py2.py3-none-any.whl (1.2 kB)
Installing collected packages: bs4
Successfully installed bs4-0.0.2
```

Dataset Preparation

Read Data

```
# Load the data
!wget -nc https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_Office_Products_v1_00.tsv.gz
```

```
--2026-01-29 09:01:23--  https://web.archive.org/web/20201127142707if_/https://s3.amazonaws.com/amazon-reviews-pds/tsv/amazon_reviews_us_Office_Products_v1_00.tsv.gz
Resolving web.archive.org (web.archive.org)... 207.241.237.3
Connecting to web.archive.org (web.archive.org)|207.241.237.3|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 512323500 (489M) [application/x-gzip]
Saving to: ‘amazon_reviews_us_Office_Products_v1_00.tsv.gz’
```

```
amazon_reviews_us_O 100%[===================>] 488.59M  24.2MB/s    in 22s

2026-01-29 09:01:54 (22.2 MB/s) - ‘amazon_reviews_us_Office_Products_v1_00.tsv.gz’ saved [512323500/512323500]
```

```
# Read the data
df = pd.read_csv('amazon_reviews_us_Office_Products_v1_00.tsv.gz', sep='\t', on_bad_lines='skip', low_memory=False)
```

Keep Reviews and Ratings

```
# Converting 'star_rating' to numeric and any invalid or non-numeric values will be turned into NaN
df['star_rating'] = pd.to_numeric(df['star_rating'], errors='coerce')

# Dropping rows where 'star_rating' or 'review_body' is missing/invalid
df.dropna(subset=['star_rating', 'review_body'], inplace=True)

# Converting star ratings to integers (e.g., 1.0 -> 1)
# This makes it easier to compare rating values
df['star_rating'] = df['star_rating'].astype(int)

print("--- Sample Reviews and Ratings ---")
print(df[['review_body', 'star_rating']].head(3))
print("\n")

print("--- Statistics of the Ratings ---")
rating_stats = df['star_rating'].value_counts().sort_index()
for rating, count in rating_stats.items():
    print(f"Rating {rating}: {count} reviews")
print("\n")

# BEFORE dropping the neutral reviews
# Positive reviews (ratings greater than 3)
positive_count = len(df[df['star_rating'] > 3])
# Negative reviews (ratings 2 or lower)
negative_count = len(df[df['star_rating'] <= 2])
# Neutral reviews (rating equal to 3)
neutral_count = len(df[df['star_rating'] == 3])

print("--- Count of reviews ---")
print(f"Positive reviews: {positive_count}")
print(f"Negative reviews: {negative_count}")
print(f"Neutral reviews: {neutral_count}")

# Drop all reviews with a star rating of 3
# Neutral reviews are excluded to simplify binary classification
df = df[df['star_rating'] != 3]
```

```
--- Sample Reviews and Ratings ---
                                     review_body  star_rating
0                                  Great product.            5
1  What's to say about this commodity item except...            5
2    Haven't used yet, but I am sure I will like it.            5
```

```
--- Statistics of the Ratings ---
Rating 1: 306967 reviews
Rating 2: 138381 reviews
Rating 3: 193680 reviews
Rating 4: 418348 reviews
Rating 5: 1582704 reviews


--- Count of reviews ---
Positive reviews: 2001052
Negative reviews: 445348
Neutral reviews: 193680
```

Relabeling and Sampling

```
# Creating Binary Labels
# 1 = Positive (>3), 0 = Negative (<=2)
# Since we already dropped 3s, everything remaining is either >3 or <=2
df['label'] = df['star_rating'].apply(lambda x: 1 if x > 3 else 0)

# Displaying the first few rows to verify the labels
print(df[['star_rating', 'label']].head())

# Downsampling
# Randomly selecting 100,000 positive and 100,000 negative reviews
pos_reviews = df[df['label'] == 1].sample(n=100000, random_state=42)
neg_reviews = df[df['label'] == 0].sample(n=100000, random_state=42)

dataset = pd.concat([pos_reviews, neg_reviews]).sample(frac=1, random_state=42).reset_index(drop=True)

# Average Length (Before Cleaning)
dataset['len_pre_clean'] = dataset['review_body'].apply(len)
print(f"Average length before cleaning: {dataset['len_pre_clean'].mean():.4f}")
```

```
   star_rating  label
0            5      1
1            5      1
2            5      1
3            1      0
4            4      1
Average length before cleaning: 318.0072
```

Data Cleaning

```
# Contraction Dictionary
contraction_dict = {
    "won't": "will not",
    "can't": "cannot",
    "n't": " not",
    "'re": " are",
    "'s": " is",
    "'d": " would",
    "'ll": " will",
```

```
        "'t": " not",
        "'ve": " have",
        "'m": " am",
        "i'm": "i am",
        "aren't": "are not",
        "couldn't": "could not",
        "didn't": "did not",
        "doesn't": "does not",
        "don't": "do not",
        "hadn't": "had not",
        "hasn't": "has not",
        "haven't": "have not",
        "he'd": "he would",
        "he'll": "he will",
        "he's": "he is",
        "i'd": "i would",
        "i'll": "i will",
        "i've": "i have",
        "isn't": "is not",
        "it's": "it is",
        "let's": "let us",
        "mightn't": "might not",
        "mustn't": "must not",
        "shan't": "shall not",
        "she'd": "she would",
        "she'll": "she will",
        "she's": "she is",
        "shouldn't": "should not",
        "that's": "that is",
        "there's": "there is",
        "they'd": "they would",
        "they'll": "they will",
        "they're": "they are",
        "they've": "they have",
        "we'd": "we would",
        "we're": "we are",
        "we've": "we have",
        "weren't": "were not",
        "what'll": "what will",
        "what're": "what are",
        "what's": "what is",
        "what've": "what have",
        "where's": "where is",
        "who'd": "who would",
        "who'll": "who will",
        "who're": "who are",
        "who's": "who is",
        "who've": "who have",
        "wouldn't": "would not",
        "you'd": "you would",
        "you'll": "you will",
        "you're": "you are",
        "you've": "you have"
    }

    def expand_contractions(text):
```

```
        # Using regex to ensure we match whole words or suffixes correctly
        # specific patterns like "won't" need to be checked before "n't"
        for contraction, expansion in contraction_dict.items():
            text = re.sub(r"\b" + re.escape(contraction) + r"\b", expansion, text)
        return text

    def clean_text(text):
        # converting to lowercase
        text = str(text).lower()

        # Removing HTML
        if "<" in text:
            text = BeautifulSoup(text, "html.parser").get_text()

        # Removing URLs
        text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

        # Expand Contractions
        text = expand_contractions(text)

        # Removing non-alphabetical characters
        # Keeping spaces so words don't merge
        text = re.sub(r'[^a-z\s]', ' ', text)

        # Removing extra spaces
        text = re.sub(r'\s+', ' ', text).strip()

        return text

    dataset['cleaned_reviews'] = dataset['review_body'].apply(clean_text)

    # Average Length (After Cleaning)
    dataset['len_post_clean'] = dataset['cleaned_reviews'].apply(len)
    print(f"Average length after cleaning: {dataset['len_post_clean'].mean():.4f}")
```

```
Average length after cleaning: 303.3003
```

Pre-processing

Removing the stop words and performing lemmatization

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess_text(text):
    words = text.split()
    # Remove stopwords and lemmatize
    filtered_words = [
        lemmatizer.lemmatize(word.lower())
```

```
        for word in words
        if word.lower() not in stop_words
    ]
    return " ".join(filtered_words)

dataset['preprocessed_reviews'] = dataset['cleaned_reviews'].apply(preprocess_text)

print("--- SAMPLE REVIEWS (Before & After) ---")

for i in range(3):
    print(f"Review {i+1} - Before:")
    print(dataset['cleaned_reviews'].iloc[i])
    print(f"\nReview {i+1} - After:")
    print(dataset['preprocessed_reviews'].iloc[i])
    print("\n" + "-"*50 + "\n")

# Length BEFORE preprocessing (characters)
dataset['len_before'] = dataset['cleaned_reviews'].apply(len)

# Length AFTER preprocessing (characters)
dataset['len_after'] = dataset['preprocessed_reviews'].apply(len)

print("--- REVIEW LENGTH STATISTICS ---")
print(f"Average length before preprocessing: {dataset['len_before'].mean():.4f}")
print(f"Average length after preprocessing:  {dataset['len_after'].mean():.4f}")
```

```
--- SAMPLE REVIEWS (Before & After) ---
Review 1 - Before:
this ink did not work on my epson wf printer would not recognize and therefore would not print tried various times rebooted printer etc the customer service at amazon were amazing in their

Review 1 - After:
ink work epson wf printer would recognize therefore would print tried various time rebooted printer etc customer service amazon amazing understanding issued refund inconvenient busy buyer

--------------------------------------------------

Review 2 - Before:
these chalk markers have a great medium sized tip and erase well with a wet cloth so far i am loving them and i hope they last throughout the school year

Review 2 - After:
chalk marker great medium sized tip erase well wet cloth far loving hope last throughout school year

--------------------------------------------------

Review 3 - Before:
the ink runs out too fast not very good product

Review 3 - After:
ink run fast good product

--------------------------------------------------

--- REVIEW LENGTH STATISTICS ---
Average length before preprocessing: 303.3003
Average length after preprocessing:  187.2127
```

Bigram Feature Extraction

```
vectorizer = TfidfVectorizer(ngram_range=(2, 2))

# Fit and transform
X = vectorizer.fit_transform(dataset['preprocessed_reviews'])
y = dataset['label']

# 80% Train, 20% Test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Perceptron

```
def train_and_eval(model, model_name):
    # Train
    model.fit(X_train, y_train)

    # Predict
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    def print_metrics(split_name, y_true, y_pred):
        acc = accuracy_score(y_true, y_pred)
        prec = precision_score(y_true, y_pred)
        rec = recall_score(y_true, y_pred)
        f1 = f1_score(y_true, y_pred)

        print(f"{model_name} {split_name} Accuracy: {acc:.4f}")
        print(f"{model_name} {split_name} Precision: {prec:.4f}")
        print(f"{model_name} {split_name} Recall: {rec:.4f}")
        print(f"{model_name} {split_name} F1-score: {f1:.4f}")

    print_metrics("Training", y_train, y_train_pred)
    print_metrics("Testing", y_test, y_test_pred)

perceptron = Perceptron(random_state=42)
train_and_eval(perceptron, "Perceptron")
```

```
Perceptron Training Accuracy: 0.9689
Perceptron Training Precision: 0.9975
Perceptron Training Recall: 0.9400
Perceptron Training F1-score: 0.9679
Perceptron Testing Accuracy: 0.8363
Perceptron Testing Precision: 0.8614
Perceptron Testing Recall: 0.8034
Perceptron Testing F1-score: 0.8314
```

SVM

```
svm = LinearSVC(random_state=42)
train_and_eval(svm, "SVM")
```

```
SVM Training Accuracy: 0.9911
SVM Training Precision: 0.9839
SVM Training Recall: 0.9986
SVM Training F1-score: 0.9912
SVM Testing Accuracy: 0.8760
SVM Testing Precision: 0.8491
SVM Testing Recall: 0.9161
SVM Testing F1-score: 0.8813
```

Logistic Regression

```
logreg = LogisticRegression(random_state=42, max_iter=1000)
train_and_eval(logreg, "Logistic Regression")
```

```
Logistic Regression Training Accuracy: 0.9544
Logistic Regression Training Precision: 0.9470
Logistic Regression Training Recall: 0.9626
Logistic Regression Training F1-score: 0.9547
Logistic Regression Testing Accuracy: 0.8746
Logistic Regression Testing Precision: 0.8587
Logistic Regression Testing Recall: 0.8981
Logistic Regression Testing F1-score: 0.8780
```

Naive Bayes

```
mnb = MultinomialNB()
train_and_eval(mnb, "Naive Bayes")
```

```
Naive Bayes Training Accuracy: 0.9434
Naive Bayes Training Precision: 0.9779
Naive Bayes Training Recall: 0.9072
Naive Bayes Training F1-score: 0.9412
Naive Bayes Testing Accuracy: 0.8560
Naive Bayes Testing Precision: 0.8840
Naive Bayes Testing Recall: 0.8211
Naive Bayes Testing F1-score: 0.8514
```